Lab Report by Nikhil Sharma(2025H1400095P)

OpenBikeSensor Firmware Analysis

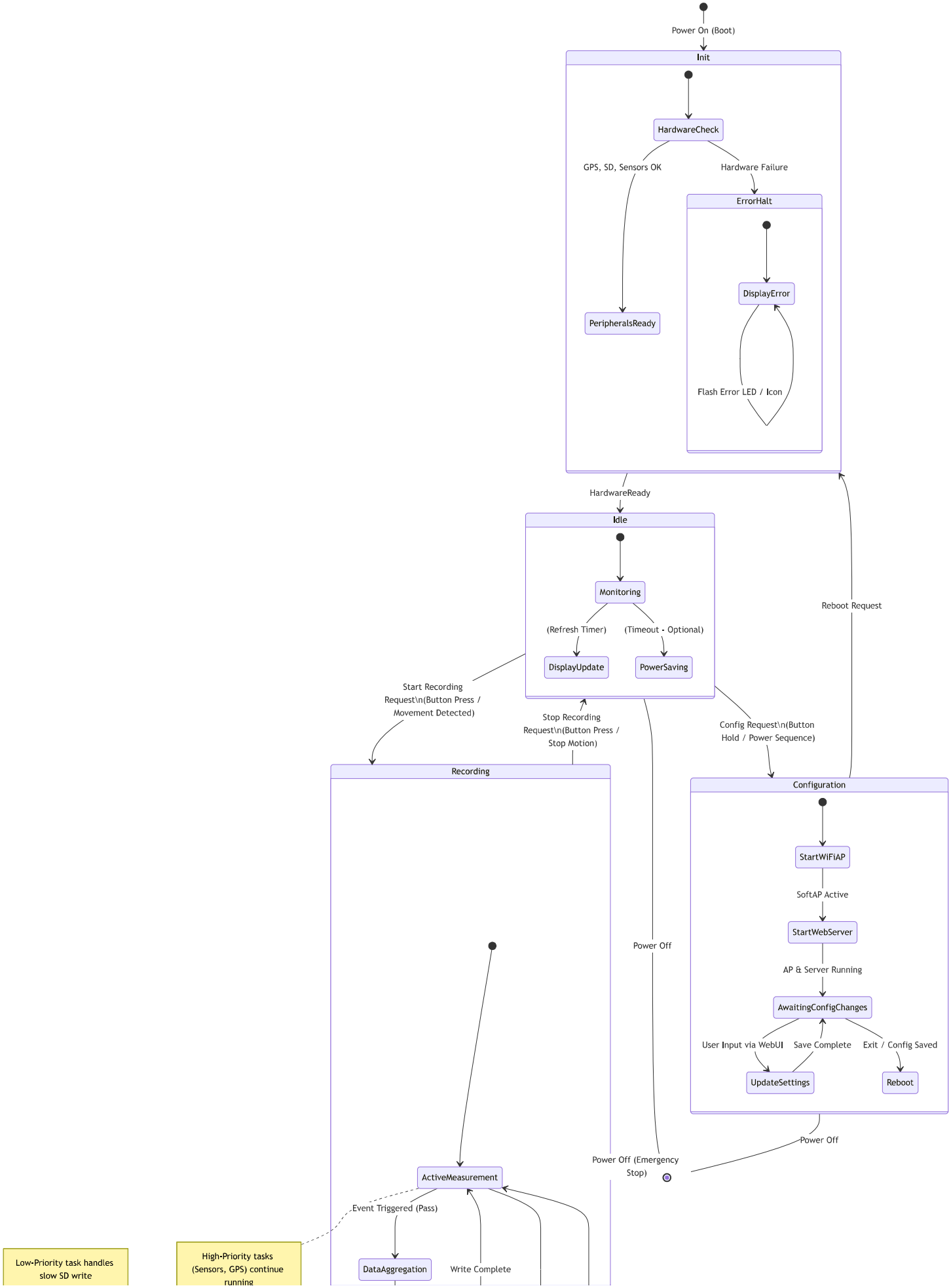
1. Behavioral Model

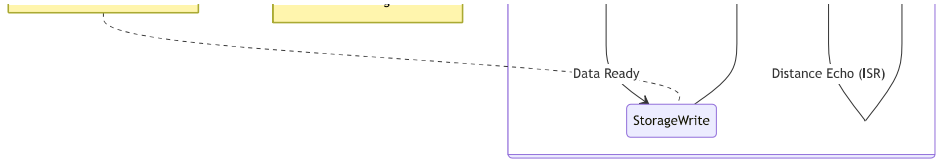
The OpenBikeSensor (OBS) firmware manages its complex responsibilities by combining Preemptive Multitasking with a Centralized State Machine. This approach ensures that high-speed sensor readings never interfere with slow background tasks like file I/O or wireless communication.

Instead of a monolithic machine, the firmware employs secondary concurrent state machines for each functional module (e.g., GPS, Sensors, UI). The global state machine transitions between the following core states based on event-driven signals:

- **INIT:** Power-on self-test, peripheral initialization.
- **IDLE:** Device is on, monitoring background status, waiting for active ride.
- **RECORDING:** The active logging state, aggregating coordinate points and sensor echo timings in a loop.
- **CONFIG (Service Mode):** Sensors power off, creating a Wi-Fi Access Point to serve configuration changes via a Web UI.

Below is a statechart reflecting the primary operational logic of the device:


Power On (Boot)
Init
HardwareCheck
GPS, SD, Sensors OK
Hardware Failure
ErrorHalt
DisplayError
Flash Error LED / Icon
PeripheralsReady
HardwareReady
Idle
Monitoring
(Refresh Timer)
(Timeout - Optional)
DisplayUpdate
PowerSaving
Reboot Request
Start Recording Request\n(Button Press / Movement Detected)
Stop Recording Request\n(Button Press / Stop Motion)
Config Request\n(Button Hold / Power Sequence)
Recording
Configuration
StartWiFiAP
SoftAP Active
StartWebServer
AP & Server Running
AwaitingConfigChanges
User Input via WebUI
Save Complete
Exit / Config Saved
UpdateSettings
Reboot
Power Off
Power Off
Power Off (Emergency Stop)
ActiveMeasurement
Event Triggered (Pass)
DataAggregation
Write Complete
Low-Priority task handles slow SD write
High-Priority tasks (Sensors, GPS) continue running



1.2 Timing and Process Synchronization

Process synchronization is critical in maintaining the ultrasonic time-of-flight measurements, where hardware latency directly translates into measurement error. To manage concurrent access collisions without breaking time-sensitive logic, the system utilizes:

- **Mutexes (Task-to-Task):** Shared peripherals like I2C buses or shared data structures (e.g., unified GPS location structs) are mutually locked so background tasks reading data do not crash against tasks writing data.
- **Critical Sections (Task-to-ISR):** For interactions where standard Mutexes cannot be utilized (such as inside a hardware callback), `portENTER_CRITICAL()` suspends the RTOS scheduler and guarantees atomic instruction operation free from other task interruptions.
- **Event Groups:** Implements an observer pattern. Bit-flags sync states seamlessly without constant software loop polling (e.g., waking the display task natively precisely when a "GPS_HAS_FIX" flag flips).
- **FreeRTOS Queues:** To entirely bypass race conditions, the architecture relies heavily on passing data by value to thread-safe queues. This zero-shared-state buffering handles severe latency variations between microsecond ultrasonic echoes and millisecond SD Flash erasures.

2. Architecture

The OpenBikeSensor leverages the foundational ESP-IDF (Espressif IoT Development Framework) explicitly tied to the ESP32 Xtensa architecture, breaking from rigid statechart behaviors to embrace a modular layer stack structure to continuously observe passing vehicle proximity while generating a captive wireless configuration interface.

2.1 Execution Model

The system abandons the traditional Foreground/Background (Superloop + ISR) model found natively in Arduino, completely depending on an **RTOS-Driven (FreeRTOS) Multitasking Model**.
Instead of one blocking superloop, logic consists of preemptively scheduled self-contained Tasks: High-priority tasks parse the GPS UART queues and calculate rigorous time-of-flight distance variables, while asynchronous best-effort tasks operate OLED drawing and File I/O when CPU loops are otherwise idle.

2.2 Layering, Interfaces, and Encapsulation

The architecture executes via heavily abstracted vertical layers:

- **Application Layer (Logic & Coordination):** High-level event-oriented domain controllers for State Management, Unified Event Filtering, and Display Orchestration without touching direct silicon states.
- **Service & Middleware Layer:** Task priorities, Non-Volatile internal parameters, Network handling, and Thread-Safe FreeRTOS queues handling all message passing.
- **Hardware Abstraction Layer (HAL):** Device-specific driver logic wrapping the hardware interfaces, transforming application calls into native protocols (e.g., executing precise RMT functions for ultrasonic triggering or UART stream parsers).
- **Hardware Layer:** Complete array of underlying hardware peripherals including HC-SR04/JSN-SR04T distance sensors, NEO-8M GNSS receivers, Li-Po Battery outputs, and momentary input switches.

3. Implementation Details

Encapsulation dictates the majority of the Object-Oriented design mechanisms seen in the source code. Every physical peripheral is wrapped into an independent C++ class keeping UART handles and logic flags inherently private.
Global singleton patterns restrict domain processes to using dedicated global accessors ensuring multiple domain specific controllers don't attempt to manipulate hardware concurrently.

3.1 End-to-End Data Flow

The flow of an overtaking event is processed bottom-up via Producer-Consumer models spanning these independent threads:

- **Trigger (Sense Process):** The Sensor Task executes continuously pinging ultrasonic edges. If an echo return reveals a sub-threshold proximity measurement, it is routed immediately to a Central Event Queue via ISR.
- **Context (Sync Process):** Independently, the GPS Task persistently cleans NMEA lines updating global Shared Memory structs with the most recent precision latitude and UTC time, protected via a system Mutex.
- **Coordinator (Fuse Process):** An application level fusion loop detects the queue's passing distance and extracts the time-sync values from the GPS struct, successfully validating the merge into a structured DataSet event.

- **Logger (Commit Process):** Utilizing a Logging FreeRTOS Queue acting as a write buffer, the system commits DataSets in CSV formatting onto the slower SPI SD storage as an asynchronous operation without freezing active tracking.

3.2 Portability and Hardware

Portability Characteristics

While the software maintains extremely high portability across custom internal OBS PCB board layouts (managed neatly via macro configurations routing C++ polymorphism patterns to different variants of sensors/GPS drivers), the project exercises virtually zero platform-level portability. The codebase requires complete rewrites to abandon ESP32 architecture because of the hard lock-in to native FreeRTOS allocations, the ESP-IDF TCP/IP Wi-Fi integration, and highly specific bespoke hardware features such as the proprietary RMT registers used for timing.

Power Management and Sleep States

Hardware strategies decouple pure quiescent draw via active toggle switches, paired with accurate low-dropout voltage regulators. On the software end, aggressively disabling Wi-Fi/BLE nodes limits background RF drain. The FreeRTOS scheduler initiates `Wait For Interrupt` logic automatically idling the CPU. Deep sleep, RAM clock-gating, and dynamic frequency scaling are managed natively by ESP-IDF integrations.

4. Caveats, Drawbacks, and Improvements

A review of the firmware's design constraints highlights limitations connected directly to platform selection:

- **Radio Priority Contention:** Closed-source Wi-Fi blobs force proprietary interruptions that have the potential to mask and jitter high-priority software timer measurements rendering small errors within the sensor logic.
- **SD Card SPI Bottlenecks:** Although FreeRTOS mitigates slow storage, cheap consumer flash drives stalling mid-write cycle can overflow the logger queue, leading directly to unregistered vehicle datasets.
- **High Baseline Power Consumption and Heap Management:** ESP32 baseline current overhead coupled with extensive Dynamic Memory allocations necessary for serving Wi-Fi JSON arrays can gradually lead to heap decay.

Improvements & Refactoring Pathways

Current reliance on Software ISRs for timing echo delays could be critically improved by offloading timestamps completely across the software boundary utilizing specific hardware Input Capture Timers / DMA, neutralizing RF stutter interference entirely.

Furthermore, substituting rudimentary ping validations with algorithmically intense noise-rejection equations, such as sliding window median filters mathematically cross-checking valid vectors natively in C++, would prevent phantom wind turbulence from registering as false-positive vehicle passes.